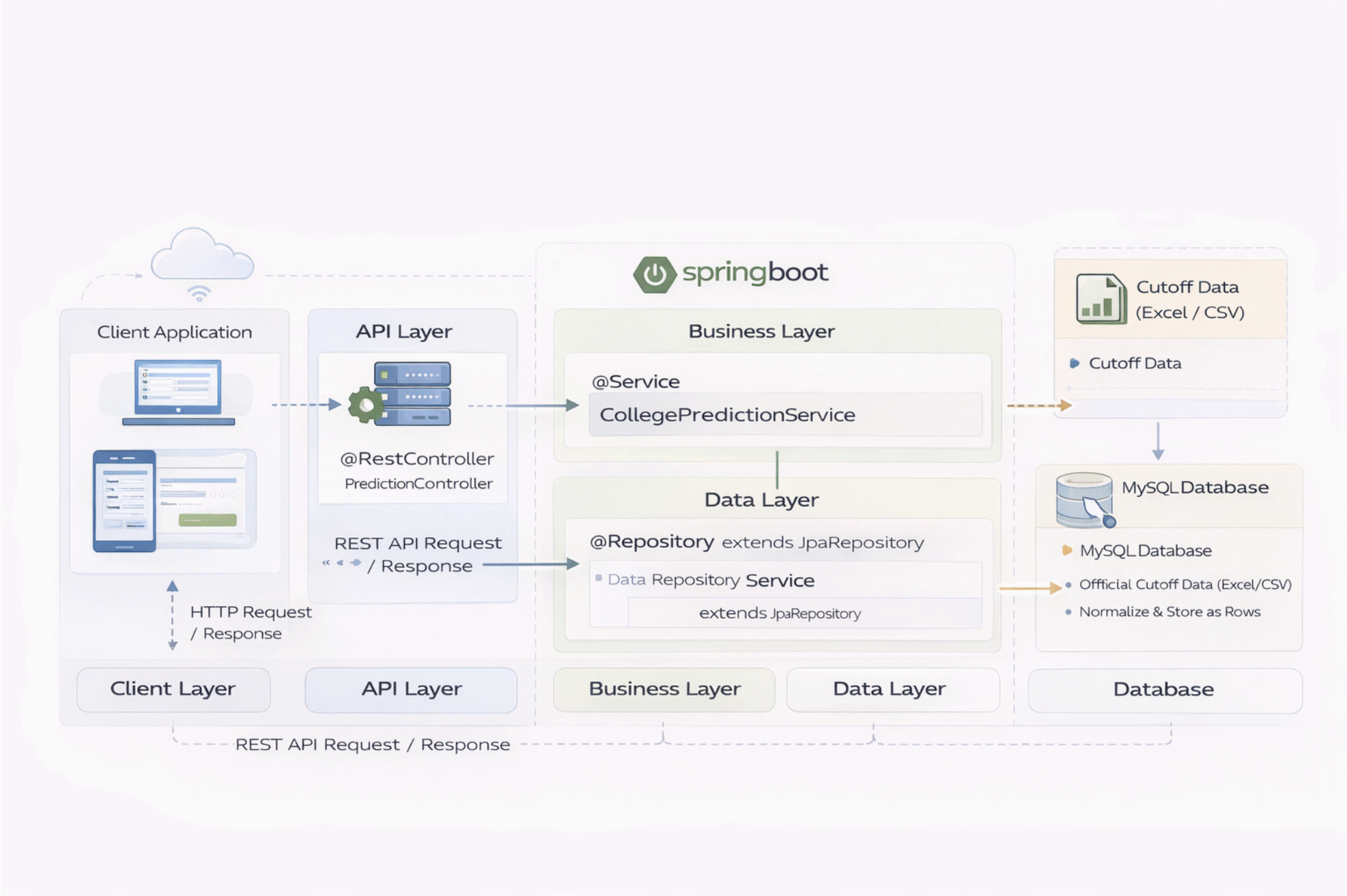


College Predictor

1. Introduction

The College Rank Predictor is a web-based application developed to help students predict eligible colleges and branches based on their entrance exam rank. The system uses historical cutoff data stored in a MySQL database and applies eligibility logic through a Spring Boot backend. The frontend is developed using React.

2. Background



Students appearing for engineering entrance examinations (such as TGEAPCET, EAMCET, etc.) need to analyze previous year cutoff ranks to determine eligible college and branches. The official cutoff data is released in Excel format, making manual analysis time-consuming and error-prone.

The College Rank Predictor system automates this process by storing historical cutoff data in a structured MySQL database and providing REST API to predict eligible college based on user chosen exam,rank, category, and gender.

User Input → API → DB → Logic → Result

3.Goal of the Project

The goal of the College Predictor System is to develop a web-based application that helps students identify suitable colleges based on their entrance exam rank, category, gender, and state.

The system uses historical cutoff data stored in a structured database to provide accurate and real- time predictions through a REST-based backend service. By automating the filtering and comparison process, the application reduces the time and confusion involved in manually checking cutoff lists, ensuring a faster, reliable, and user-friendly college selection experience.

4. Functional Requirements

4.1 User Input Handling

1. The system shall accept the following inputs from the user:
 - Rank (Numeric value)
 - Category (e.g., OC, BC-A, BC-B, SC, ST, etc.)

- Gender (Male/Female)
- State/Region (OU/AU/SVU)
- Exam Name (EAMCET, TGEAPCET, etc.)

4.2 Backend API Processing

Once the user submits the form, the system shall send the input data to the backend through a RESTful API endpoint. The Controller layer shall receive the request and forward it to the Service layer for business logic execution. The backend shall process the input parameters and prepare the necessary filtering conditions. The API shall return the prediction results in JSON format.

4.3 Database Filtering and Prediction Logic

The system shall retrieve historical cutoff data from the MySQL database based on the provided exam name, state, category, and gender. The prediction logic shall compare the candidate’s rank with stored cutoff ranks and determine eligibility based on defined criteria. Only colleges and branches satisfying the eligibility conditions shall be included in the final result set.

4.4 Result Display

The system shall display the filtered results in a structured table format on the frontend. The displayed information shall include Institute Name, Branch Name, Cutoff Rank, and relevant details. If no colleges match the entered criteria, the system shall display an appropriate message indicating that no eligible colleges were found.

4.5 Multi-Exam Support

The system shall support multiple entrance examinations such as EAMCET and TGEAPCET. The selected exam shall determine which dataset is queried in the database. The system architecture shall allow future addition of new entrance exams without requiring major structural modifications.

5. API Data Model

5.1 API Endpoint

Example :

```
GET /api/predict
```

5.2 Request Data Model

The API uses **HTTP GET** with **Query Parameters**.
The system uses @RequestParam .

Query Parameter Format

```
?parameter=value&parameter=value
```

Example :

```
GET /api/predict?rank=45000&category=BC_B&gender=GIRLS&state=OU
```

5.3 Backend Controller

```
@RestController
@RequestMapping("/api")
public class CollegeController {

    @GetMapping("/predict")
    public List<CollegeCutoff> predict(
        @RequestParam int rank,
        @RequestParam String examName,
        @RequestParam String category,
        @RequestParam String gender,
        @RequestParam String state) {

        return service.predict(rank, category, gender, state);
    }
}
```

```
}
```

5.4 Parameter Binding Model

Spring Boot automatically binds query parameters to method arguments.

Example:

```
GET /api/predict?rank=45000
```

Is mapped to:

```
@RequestParam int rank
```

5.5 Internal Processing & Data Retrieval Model

After receiving the request parameters (rank , category , gender , state), the backend processes the request using the following flow:

```
Controller → Service → Data Retrieval → JSON Response
```

5.6 Response Data Model

The API returns a **JSON Array** of eligible colleges.

Example Response

```
[
  {
    "id": 1,
    "instituteName": "ABC Engineering College",
    "branchName": "CSE",
    "category": "BC_B",
    "gender": "GIRLS",
    "state": "OU",
    "closingRank": 65000
  }
]
```

5.7 API Contract Summary

```
HTTP Method    : GET
Endpoint       : /api/predict
Input Format    : Query Parameters
Output Format   : JSON Array
Content-Type   : application/json
```

6. Database Schema

The system uses a relational database (MySQL) to store college cutoff data.

Each record represents the closing rank for a specific:

1. College Name
2. Branch
3. Category
4. Gender
5. State
6. Exam Name

6.1 SQL Table Definition

```
CREATE TABLE exam_cutoffs (
  id INT PRIMARY KEY AUTO_INCREMENT,
  exam_name VARCHAR(50) NOT NULL,
  instcode VARCHAR(50) NOT NULL,
```

```
institute_name VARCHAR(500) NOT NULL,  
branch_code VARCHAR(50) NOT NULL,  
category VARCHAR(20) NOT NULL,  
gender VARCHAR(10) NOT NULL,  
closing_rank INT NOT NULL  
);
```

6.2 Normalized Data Design

Instead of storing separate columns like:

```
OC_BOYS, OC_GIRLS, SC_BOYS, SC_GIRLS
```

The system stores them in row format:

```
| exam_name | instcode | branch_code | category | gender | closing_rank |
```

This ensures:

- Flexibility
- Multi-exam support
- Easy filtering
- Scalable design

6.3 Sample Data

```
INSERT INTO college_cutoffs  
(institute_name, branch_name, category, gender, state, closing_rank)  
VALUES  
( 'ABC Engineering College', 'CSE', 'BC_B', 'GIRLS', 'OU', 46000),  
( 'ABC Engineering College', 'CSE', 'BC_B', 'BOYS', 'OU', 44000),  
( 'XYZ Institute', 'ECE', 'SC', 'GIRLS', 'AU', 78000);
```

6.4 Entity Mapping (JPA)

```
@Entity  
@Table(name = "college_cutoffs")  
public class CollegeCutoff {  
  
    @Id  
    private Long id;  
  
    private String instituteName;  
    private String branchName;  
    private String category;  
    private String gender;  
    private String state;  
    private Integer closingRank;  
}  
  
);
```

6.5 Importing Excel Data into MySQL

The data import process is as follows:

1. Open the Excel cutoff file.
2. Save the file as CSV (Comma Separated Values).
3. Open MySQL Workbench.
4. Use the “Table Data Import Wizard”.
5. Select the CSV file.
6. Map the columns correctly to the database table.
7. Complete the import process.
After import, the cutoff data is stored inside the MySQL database table.

7. System Architecture

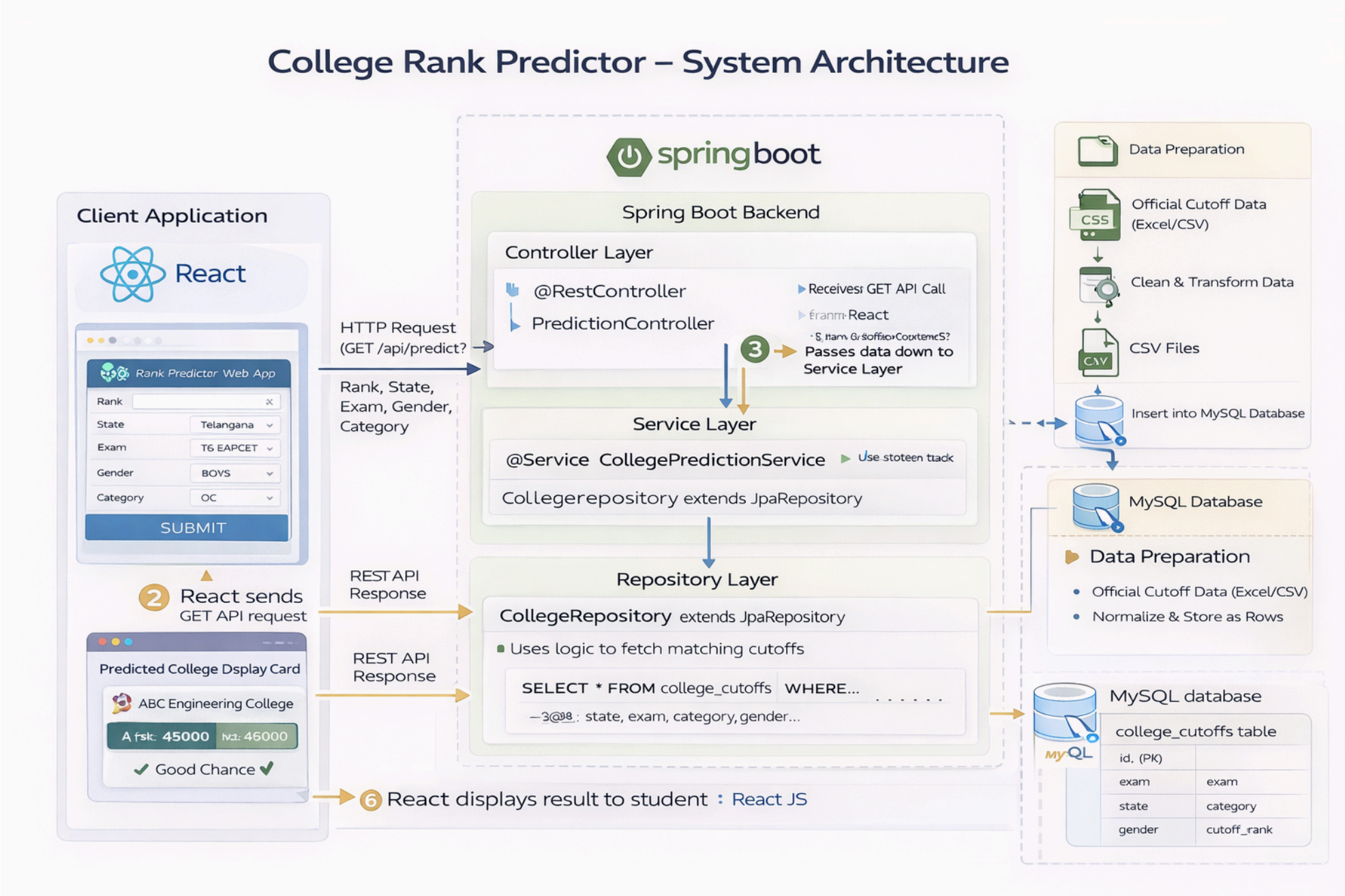
7.1 Overview

The College Rank Predictor follows a Three-Tier Architecture consisting of:

- 1. Presentation Layer (Frontend – React)
- 2. Application Layer (Backend – Spring Boot)
- 3. Data Layer (MySQL Database)

The architecture ensures separation of concerns, scalability, and maintainability.

7.2 High-Level Architecture Diagram



7.3 Architectural Components

Presentation Layer (React)

Responsibilities:

- Collect user input (rank, exam, category, gender)
- Call backend REST API
- Display prediction results
- Dynamically fetch categories based on selected exam
- Technology Used:
- React JS
- Fetch / Axios

7.4 Application Layer (Spring Boot)

This layer contains:

a) Controller Layer

- a) Handles HTTP requests
- b) Maps query parameters
- c) Returns JSON responses

Example:

```
GET /api/predict
```

b) Service Layer

- Implements prediction logic
- Filters data based on:
 -
 - exam_name
 - category
 - gender
 - closing_rank >= user_rank
- Sorts results

c) Repository Layer

```
a) Interacts with database
b) Uses JPA / SQL queries
c) Retrieves cutoff data efficiently
```

Data Layer (MySQL)

Stores historical cutoff data in normalized format.
Table:

```
exam_cutoffs`
```

Each row represents:

- One exam
- One college
- One branch
- One category
- One gender
- One closing rank
- To optimize query performance.

7.4 Request Flow

When a user submits the form :

```
User → React → REST API → Service → Database → Service → JSON → React → UI
```

Step-by-step:

```
1) User enters rank and selects exam.
2) React sends GET request to backend.
3) Controller receives request.
4) Service applies filtering logic.
5) Repository fetches matching rows.
6) Results returned as JSON.
7)React displays eligible college.
```

7.5 Scalability Considerations

The architecture supports:

- Adding new exams without schema changes
- Adding new categories dynamically
- Handling large datasets through indexing
- Horizontal scaling of backend if required

7.6 Extensibility

New features can be added easily:

- New entrance exams
- ML-based prediction
- College ranking integration
- Admin upload portal

Without modifying core architecture.